Fediflock: A Federated Network for Multi-Agent Systems using Virtual Pets

June Bhartia
M2 IGD - Institut Polytechnique de Paris

***Abstract*—Virtual pets are an interesting technology to study as they help us understand how humans treat and interact with artificial creatures that they perceive to be alive. Despite virtual pets existing for more than 3 decades, there has been little literature on inter-pet interaction methods. This thesis explores autonomous pet models and decentralized communication systems for interaction between multiple virtual pets. The work proposes and implements Fediflock, a framework that models pets, environments, and activities, and a decentralized communication mechanism that allows pets to discover each other and engage in shared activities. To evaluate the work, a user study with developers was conducted. The study looked at how developers understand and want to extend components of the framework. The results indicate that participants generally found the simulation understandable. Extension suggestion themes include increasing simulation complexity, connecting to existing social platforms, porting to new devices, and discussions on use of the environment. These findings suggest that the Fediflock framework provides a foundation for developing interconnected virtual pets. Beyond the framework, the work also highlights that studying virtual pets' interaction methods can help us study online social connections and imagine pets that aren't confined to singular environments.**

I. Introduction

Neko [1], a desktop accessory, was released in 1988 as software that makes a virtual cat follow the user's cursor on the screen, making it one of the first known virtual pets. Since then, we have had thousands of different versions and implementations of virtual pets such as Tamagotchi [2] and Neopets[1]. They come with a variety of functionalities, aesthetics, and media. Some Desktop pets distributed on Itch.io[2] emulate anime-esque characters or popular media creatures like Pokémon. Other virtual pets can be just animals on the screen or take the form of an idle game. A quick search on Itch.io or the Steam database [3] shows the variety of ways developers and designers have re-imagined these software-simulated creatures.

Virtual pets are software-simulated creatures that allow a user to interact with the system as if it were a real pet. They can have simulated stats, behaviors, or requirements. Many companion pets rely on the user's actions to 'live'. They can be defined by their autonomy, parallel existence with the user, and 'pet-likeness' [3]. They inspire a sense of real creatures in the user [3], who can form emotional bonds with these virtual pets and treat them as real companions.

[1]https://www.neopets.com/

[2]https://itch.io is a platform for sharing games and game-like media

[3]https://store.steampowered.com/

This makes them an interesting technology to study how people interact with virtual creatures that they consider a companion or something they can take care of. Virtual pets have had an established cultural impact on entertainment, which has been widely studied. They have also been shown to be useful outside entertainment and companionship, including healthcare [4], education [5], productivity tools, and habit-building practices [6] [7].

The core focus of this thesis is the design and technical aspects of virtual pets. This thesis studies how to build a virtual pet framework that is extendable by other developers, allowing them to create their own pet(s) easily. The thesis's secondary focus is on the interaction between pets and users, and the interaction between pets and other pets, which is seldom studied in the literature. This serves to address the challenges often faced by virtual pet designs, the users of the virtual pets, and the communities surrounding the pets.

A. User and Pet agency in online virtual pets

Users often form a real bond with virtual pets. However, due to the closed-source nature of many web-based virtual pets, there have been several instances of such pet hosting websites being discontinued. Leaving users to grieve their connection to the virtual creature. The ownership of the virtual pets in these cases doesn't rest in the hands of the user, but in the hands of the website that they live on, taking away agency from the user.

Pets have a social aspect to them; real dogs interact with each other in a dog park let the owners meet new people and form friendships [8]. Virtual pets that are part of a larger online world, similarly, have a social aspect to them. Users participate in that online world of the virtual pet and can form communities surrounding the game and the pet [9]. However, unlike real-life pets, these online pet social interactions are not due to the interaction between two pets, but due to a common world the users participate in together. These social interactions often rely on user-defined interactions, using the pet as a digital avatar to explore the world. They rarely allow an autonomous pet to make independent decisions to act. This may make the pet feel less independent as a creature.

Another issue is that the sites that host these digital worlds often also operate on a user-account basis for having virtual pets. If a user does not log in for a long period of time, these abandoned pets that rely *only* on that user are then helpless to do anything else. Since these virtual pets are supposed to be perceived as 'real' creatures, being stuck in eternal

hunger states or dying due to lack of interaction reduces their perceived realness. If a pet does not exist outside the interaction it has with a user, it is difficult to perceive it as independent or autonomous. This model of interaction with a virtual pet takes away agency from the pet.

B. Contributions and research question

This thesis proposes and implements a framework for a decentralized open-source network for autonomous multi-agent systems like virtual pets. The design aims to provide a way for developers to keep the social spirit of older virtual pets while allowing them to develop and integrate newer technology and have more complex internal systems. The scope of the thesis is limited to the pet model and the communication framework and does not include network optimization or security measures.

This thesis aims to explore the research questions: **RQ1:** How do we model virtual pets for autonomous behaviour and inter-pet interactions? **RQ2:** How do we design decentralized interaction strategies for virtual pet communication? **RQ3:** How do developers want to use and extend a framework for virtual pets and decentralized communication?

This thesis was done during a six-month internship at the University of Tsukuba, with Prof. Claus Aranha as supervisor. The topic was initially proposed during a Dagstuhl Seminar [10] by Prof. Claus Aranha. The process of completing the thesis entailed literature reviews on virtual pets and multi-agent systems, multiple discussions on research direction and finding research questions, prototyping and studying decentralized communication technologies, design and development of the final framework, user studies with developers, and analysis of the user study.

The research contributions of this work are :

1) A decentralized asynchronous communication model for virtual pets.
2) A framework modeling virtual pets as a self-directed multi-agent system using the above communication model.
3) Findings from a user study on how developers extend and use the framework.
4) Findings from interviews with developers and designers on use cases of the framework.

Thesis Structure: Section II provides the necessary background describing virtual pets, multi-agent systems, and decentralized communication systems and how the literature informs my work. Section III describes the framework: the different layers and how they connect to each other, the decentralized communication model for virtual pets, and the simulation model for the pets. Section IV details the validation approach, including a network usage analysis along with a two-part user study with developers. The first part tests the extensibility and usability of the framework; the second part tests the approachability of extension use cases and includes a preliminary interview on how developers want to make interactive virtual pets. Section V analyzes the results. Section VI reflects on the challenges, failed approaches, and discusses use cases, giving a qualitative overview of directions that the developers want to build towards for virtual pets and how Fediflock may fit into that. The thesis concludes with a discussion of potential directions and limitations.

II. Preliminaries

A. Virtual pets

Humans can form emotional connections with any object or creature. This emotional investment in the object can be perceived as a pet-like connection, especially in the case of virtual pets if the object is programmed to be a pet [3]. Mival et al. highlight the ”useful-uselessness” of a pet, stating that a pet does not serve a function besides being an autonomous and non-specific companion, mentioning that ”the purpose of a cat is to be a cat” [11]. This makes it difficult to define a pet, let alone a virtual pet. One such attempt was made by Dodd et al., where they describe the scope of virtual pets as *autonomous software simulated creatures that ultimately rely on the user* [12]. Therefore, virtual pets can be described in their relationship with the user. In a Dagstuhl discussion about virtual pet technologies, a pet's parallel existence with humans was mentioned as a property of virtual pets [10]. This parallel existence can give the user a perception of independence in the virtual pet.

When discussing the definitions of virtual pets, it is important to distinguish them from artificial companions. Artificial companions can provide a variety of functions and use cases, but Rogge et al. differentiated a virtual pet from an artificial companion by describing one of the applications of an artificial creature as a pet (or childlike companion) [13]. Thus putting a virtual pet as a use case or functionality of an artificial companion.

The primary domain of study for this thesis is virtual pets; hence, it becomes important to explicitly define the definition of virtual pets used in this work and what role they would play for the user. Virtual pets are often characterized by their behaviors, so this work defines them in terms of their behavior and relationship to the user.

Using the literature described above, this thesis defines a virtual pet as:

> *A software-simulated creature, autonomous and persistent, that may or may not rely on or be the responsibility of a user.*

Where persistence means

> *The capability of an autonomous system to exist, observe, and act even when the user is not actively engaging with it. They may have interactions with their virtual worlds and their inhabitants.*

B. Human and social aspect of virtual pets

A virtual pet can be distinguished by its functional role in the user's life. Similarly, the social aspect of a perceived independent creature influences the design of the software underlying it. A study in virtual pet trends by Bylieva et al. suggests using touch, game, communication, and social

interaction to analyze human-virtual pet relationships [14]. Online pets like Neopets have been shown to have socio-political, economic, and cultural impact within the online community that they are part of [9] [15]. Virtual pets can hence be studied as virtual avatar-like proxies of the users in the larger game of the online social community, which is similar to how dogs act as proxies for owners to have social interactions with other dog owners [8]. Even physically based virtual pets like Tamagotchi can enable social interactions; Tamagotchi Friends (2014) allows two toy users to interact by sending messages when touching the toys together [14].

The thesis hence proposes a framework that aims for open-source, decentralized pet systems that can communicate with each other, allowing the users to engage with the pet communications as a proxy for the user's own social engagement in the community. This aims to provide a game-like system where instead of a user's avatar, it is a pet who engages with the world, and the user perceives the world only from their interaction with the pet.

C. Multi-Agent Systems

If we consider a virtual pet as a self-directed software system, multiple pets interacting with each other can be described as a kind of Multi-Agent System. An agent can be described as a program with autonomy of making choices, social ability to interact with humans or the environment, reactivity to changes in its environment, and pro-activeness of taking actions [16]. In this definition, an environment is a place where an entity can sense different parameters and make necessary decisions with that information [17]. We can thus adapt this description of an environment to virtual pets and consider the environment as the facilitator of information needed for the decisions to be made by the pet.

The literature of Multi-Agent Systems considers decentralized systems in which multiple such agents are capable of interacting with each other and carrying out their respective goals or tasks delegated to them [18]. Design and development of multi-agent systems deals with multiple challenges, including communication and the expected societal design of the whole system. For communication, in early attempts of establishing a common language of agent-to-agent communication, Finin et. al. introduced KQML, which has a specific format and a message-handling protocol with the KQML API [19]. But generally, these communication languages are referred to as Agent Communication Languages (ACL), where agents communicate using unique message formats and ontologies, allowing them to send messages and interpret received messages [17]. In recent years, the development of Large Language Models as agents[cite] has led to protocols to enable communication for these agents; Agent2Agent (A2A) Protocol [20] enables a peer-to-peer (p2p) like protocol for such communication by giving each agent an API endpoint for communicating, allowing the user to communicate with a remote agent using a local agent.

The research in this work deals with virtual pet-to-pet communication, considering each pet as an agent. This lets us take inspiration from communication and society design for multi-agent systems.

D. Federation

Decentralized Online Social Networks (DOSNs) allow users to communicate and share information across independently operated systems without relying on a single central system. Datta et al. discuss the challenges of connecting third-party applications within these DOSNs without requiring changes from all users [21]. While Lacava et al. examines the requirements and growth of the Fediverse (a collection of decentralized social platforms). They state two aspects of this growth: the availability of open-source software to allow everyone to set up their server, and the existence of specific communication protocols to enable fluid interconnections between servers [22].

ActivityPub [23] is one such protocol that addresses interoperability between independently operated instances by defining shared data formats and communication conventions. Similarly, the AT Protocol [24] provides a decentralized architecture for social networking, using structured data schemas that can be extended to introduce new types of behaviour. Both protocols provide a basis for communication between independently operated systems while allowing applications to define their own functionality.

Other decentralized communication protocols address different requirements. Matrix [25] provides an open network for secure, decentralized communication, and has been used in a range of applications. However, challenges remain concerning security, real-time communication, synchronization, scalability, and interoperability [26]. Zenoh [27], in contrast, targets communication across networks and devices, including resource-constrained devices, by operating at the network and data link layers of the OSI model.

These protocols show how decentralized communication can be designed for different situations, from social networks to robotics. However, their different assumptions and goals mean that a protocol designed for social communication or device-to-device data exchange may not address the requirements of autonomous agents (such as virtual pets) that need to discover other agents, coordinate activities, and exchange structured state updates across independently operated servers. While the underlying protocol used by this thesis is simple, the framework is built with the above-mentioned protocols in mind. The goal is to abstract and decouple the communication system from the simulation systems. So a change in protocol would not affect the method of discovery and interaction.

III. Presentation of Work

In this thesis, I develop Fediflock, a framework for building autonomous and persistent multi-agent systems like virtual pets that need to communicate with each other and the user asynchronously. It is designed for communications that can span time and operate on a tick-based time system.

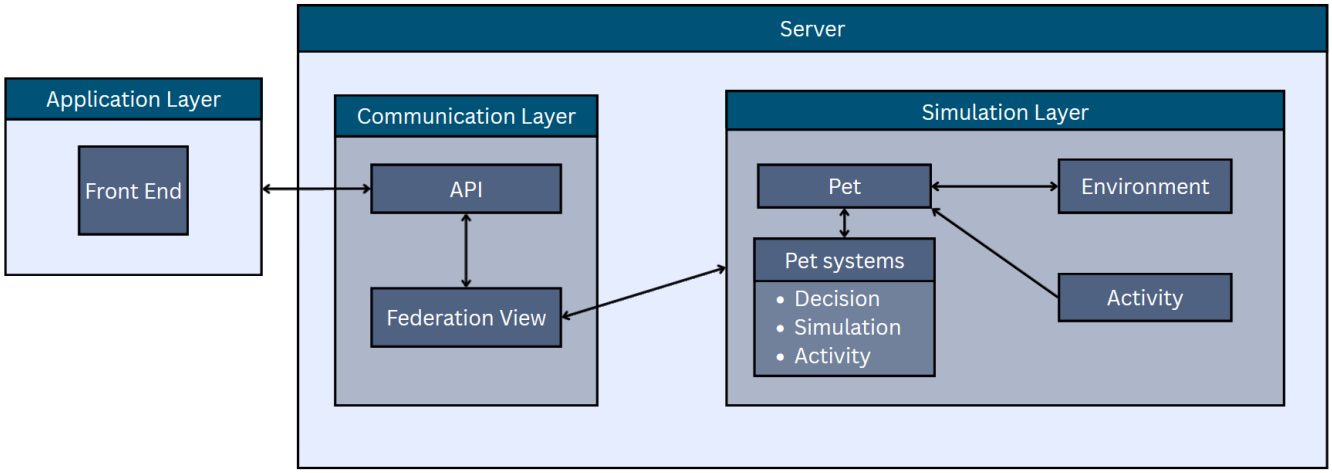


Fig. 1. The application, communication, and simulation layers in relation to the framework's objects and server.

Fediflock can be accessed on github[4]. The GitHub includes a wiki explaining the code architecture and details of classes. It also includes a link to a client-side website. The website functions as a front end for any API implementation of Fediflock, along with project explanation and installation instructions. The framework provides an API for communication between servers and to access information on the server. It includes 3 layers that allow it to abstract away lower-level tasks like making fetch calls for the API as shown in Figure 1.

The framework's primary components are the pet, activity, user, and environment models, the pet systems that make the pet autonomous, the federation abstraction for communication, and the API routes for communication. The framework also consists of *data* files to instantiate and query static data for environment or activity instantiations and *view* files for each of the models that allow for controlled and flattened sharing of information about the respective models.

The framework uses specific definitions of pets, activities, and environments that define what each model is capable of. Each of these are first level objects, meaning they exist on the server as objects that the simulation can directly access and perform operations on. The operations are usually performed by incoming API requests received by the app. The models are:

- **Pet:** An autonomous, persistent agent. It needs an environment to be present in, and has internal systems that dictate its behavior.
- **Environment:** The container for pets; each pet has to live in an environment. Pets can query the environment for information to further their goals. Two pets in the same environment can discover each other and engage in an activity. Environments can contain items that have an associated activity.
- **Activity:** Represents any activity done by one or more pets; it triggers and sends information to related entities about the activity status or ticks. An activity is the way a pet interacts with another pet or the user.

[4]https://github.com/JuneVamp/FediGotchi

A. Pet Model and Systems

The pet model contains the data necessary to define the pet and pet systems to add behavior to the pet. The systems are built to be modular additions to the model so that developers may easily add new behaviors. The important pet systems are:

- **Activity system:** The activity system manages the pet's activities, keeps track of the stats, and contains the state machine for various states of an activity initiation process. It also communicates with the activity object and receives the activity ticks or finished notifications and translates any stat changes for the simulation system.
- **Decision system:** The decision system behaves as the primary brain for the autonomy of the pet. It contains function calls that can be triggered on various threshold conditions of stats. The decision system is also responsible for choosing, accepting, or rejecting an activity or partner and thus enforcing any personality traits. (Appendix section A-C shows an example.)
- **Simulation system:** The simulation system manages the pet simulation and receives the ticks from the server (only if the pet is not doing an activity, in which case it receives ticks from the activity system). It is also in charge of processing and keeping track of stat changes either from the pet's default behavior or an activity.

A pet can find and do activities alone or with other pets (currently based on a boredom threshold).

1) A pet triggers an activity based on stat changes (boredom >30).
2) It asks the environment for items and extracts related activities, merging them with activities the pet already knows.
3) The pet randomly picks an activity and checks if the activity requires a partner.

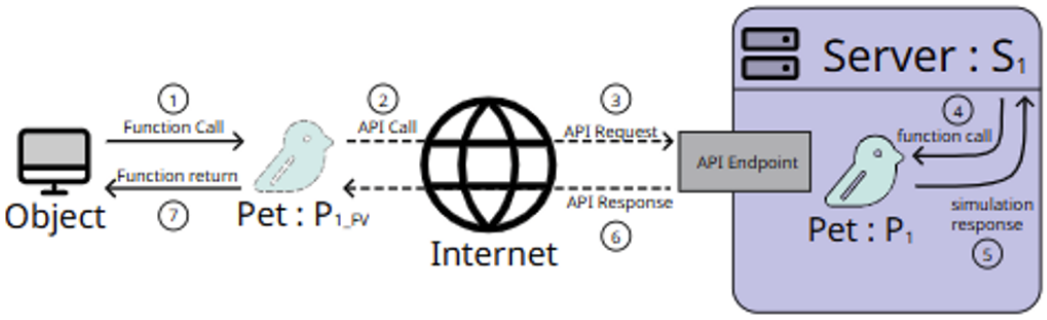


Fig. 2. The pet on the right P_1 is a local pet and the pet on the left P_{1_FV} is a remote pet's federation view Representation. A remote pet's representation can be used as if it were a local pet.

4) If the activity requires a partner, it asks the environment for other pets and chooses one randomly.
5) Once the pet has chosen an activity and maybe a partner, it will send an activity creation request to the server and send a request to the partner pet to do the activity with them.
6) If the response from the partner is positive or if the activity is solo, then the pet will send a start request to the activity.
7) The activity then sends tick updates to the pet to let the pet(s) know of activity progress.

Besides these systems, the current framework implementation also includes a constants class to instantiate different pets with different primary stats to allow randomness or different pet behaviors.

B. Environment

An interesting property of a virtual pet is that it takes up space that other programs are not expected to take; desktop pets occupy the screen, web-based pets take up screen space on the website, and Tamagotchi take up space in the real world as well as the entire screen space of the toy. This space-taking property is another factor in their perceived realness. The space they take should be able to facilitate the interactions they seek. Similar to how dog parks allow pets in the park to interact with each other. Environments for virtual pets should allow for the same interaction.

Fediflock treats the pet's container environment as the interaction facilitator space. An environment can have multiple pets. These environments are kept open to developers' design, but an environment can be any space a pet is expected to exist in, such as a virtual park, a specific area of the screen, the application window, a physical device, or a household WiFi. The environment serves as a dog park for the pet, allowing pets in the same environment to interact with each other. This would, for example, allow multiple pets on a specific application window to interact among themselves but not with pets on another application window. This model of the environment can inspire the developer to have multiple pets in an environment and design interactions for them.

An environment in the framework contains references to the pets that exist within it along with items that exist in the environment. Items functionally provide an environment with uniqueness; an item can communicate an associated activity to the pet. This activity could change the interactions that an environment allows. (Appendix section A-B shows a task where an activity mate can be triggered by a special item for mating).

C. Activity

Activity is used for all interactions and communications that take a certain amount of time to complete. This is because any interaction between two pets can be modeled as an activity. For instance, talking is an activity, and similarly, watching TV is an activity (that requires the TV item to be present in the environment).

An activity usually affects the pet's stats and can be done alone or by multiple pets at the same time. The pet's decision system can decide to trigger activities based on certain criteria; this provides the pet with the autonomy and parallel existence (as discussed in section II).

This is explicitly built to limit the one-on-one conversations that the user can explicitly participate in with the pet. Allowing complex one-on-one interactions between two pets would venture into the territory of general agents. However, communications such as activity requests behave as one-on-one communication, and the API and pet model can be easily extended to do so for other cases if needed (Appendix section A-A shows a task where two pets may need to explicitly communicate to invite one of them to the other's environment)

One major decision to make when designing a decentralized pet is to figure out who "owns" time. For example, one server may run at 1 tick per second, but another server may have 1 tick every 10 seconds. Both of these pets should still be able to interact. But then at what rate does the activity progress? This was one of the reasons to define activity as first-class objects that exist on the server instead of an extension of a

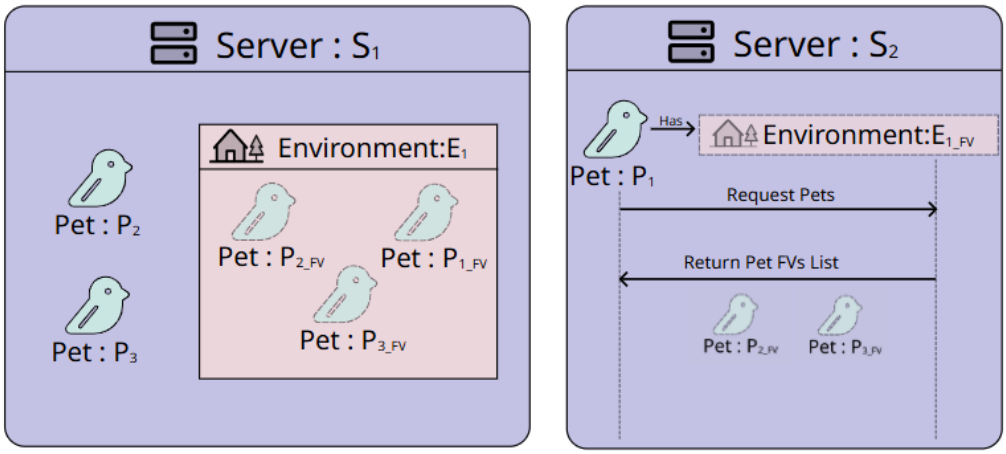


Fig. 3. The federation view allows the local pet (P_1 on server S_2) in a remote environment (E_1) to get items and pets from the remote server using the Environment's FV (E_{1_FV}). The pet P_1 can then send an activity request to the pet P_2 in that environment using the FV of that pet (P_{2_FV}).

pet's behavior. This not only allows easier access to the activity for the server but also simplifies synchronization of activities between multiple pets so events triggered by activities progress only trigger once (Appendix section A-B provides an example for an activity that can trigger another event; in this case, the mate activity would trigger creation of a new pet).

D. Communication and Federation

The framework's entities communicate using the Access Point Interface (API) Routes established in the app. These routes help communication, both internal and external, and use the internet. It provides an HTTP request endpoint that exposes parts of the simulation to the network. For example: given the server is running on `domain.com`, and pet alice on that server, making an HTTP request to `domain.com/api/pets/alice` will get the *view* of alice.

The framework proposes and implements classes called Federation Views (FV). The Federation view is a remote access point for a pet. It stores the necessary information (`id`, `serverURL` and `entityType`) to contact and establish communication with the associated entity that it is representing. It allows local objects to treat remote objects as local, allowing a programmer to call functions on the Federation View objects, abstracting the work of making HTTP calls. Each type of remote object that has an API route has a Federation View class that abstracts the HTTP calls required to call that API. Hence, each type of entity (pets, environment, activities) uses a different Federation View class.

All entities communicate with the help of the FV of the entity they are trying to communicate with. **Figure 3** shows the steps of the process of an object with the PetFV object of a pet P_1 making a function call (like `getView()`) on the PetFV (pet federation view) object:

1) The Object makes the function call on the PetFV P_{1_FV}
2) P_{1_FV} makes the HTTP call to the API for the method used in step 1 and waits for the response.
3) The server S_1 receives the request and directs it to the correct router.
4) The router determines the request is for Pet P_1 and calls the function on the object P_1
5) The P_1 processes the function call and returns anything if needed to the router
6) The router processes the response, formats it back into the appropriate API call's response, and sends the response
7) P_{1_FV} receives the response, formats the result, and sends it back to the Object that made the function call (in step 1)

IV. Method

The work is validated by a network usage analysis and a user study. This section describes the setup, study protocol, and analysis process of both validation methods.

A. Network use analysis

The framework was tested to evaluate the amount of data bandwidth and total usage. The base app monitored sent and received bytes of data using the Content-Length field of HTTP requests. During this study, the Pets are equally distributed between each available environment. So the number of pets per environment is inversely proportional to the number of environments. The framework was allowed to run by itself, varying the number of pets in each environment and varying the number of environments independently.

B. User Study

The participants were recruited based on having at least 3 months of programming experience. The user study aimed to study the extensibility, usability, and understandability of the framework. The interviews were recorded and transcribed for analysis. Before the study, the participants were sent a demographic questionnaire using Google Forms. The questionnaire had a basic consent form and collected details on their name,

gender, current profession, and domain expertise in computer science and programming. The questionnaire also asked about their familiarity with the topics of game development, game design, virtual pets, and federated networks.

The participants were briefed on the motivation and terms related to the project and then asked to install, read the wiki, and explore the front-end website using their local installation. The first task was to provide extensions or use case scenarios using the framework. The participants and the facilitator discussed their extension and use case ideas and how the framework could be helpful. The second task was to try to provide pseudocode or implementation details for 1 of the 3 possible extension tasks. The participants were allowed to choose a task from the list of three possible tasks.

The task options were:

1) **Invite** - extend the framework to allow pets to invite other pets to their environment. This involved creating an invite endpoint, updating the pet model to store contacts, and implementing functions for sending and receiving invites.
2) **Mate** - extend the framework to allow pets to reproduce and create new pets. This involved creating a mating center environment, implementing criteria for accepting or rejecting mating activities, and tracking parent-child relationships in the pet model.
3) **Personality** - extend the framework to give pets a simple personality system that influences their activity choices. This involved updating the pet model and systems to have personality traits, implementing API endpoints to fetch personality and activity history.

The goal of the task was to provide an outline of the code that would be used to implement the task; the outline must include function, class, or variable names that they wish to implement, what the piece of code does, and where it would go in the code architecture of the framework. They were informed of and encouraged to use the think-aloud protocol. The participants were encouraged to not use AI or LLM tools during the task to preserve individual human understanding of the framework for the study. They were given 45 minutes with no help besides the wiki and the task details.

After they finished the task, the facilitator conducted a 15-minute semi-structured interview which included 19 Likert scale questions inspired by SUS studies [28] and API usability studies [29]. The questions are included in the **Appendix B** Following the Likert scale questions, they were asked open-ended questions on the usability, understandability, and extensibility of the framework.

The questions were designed to test the Understandability of the code architecture(**A2, A5, A10**), Understandability of the abstractions provided by Federation views (**A4, A6**), Extensibility of the framework (**A1, A11, B1-B7**), Learnability (**A7, A8**), and usability (**A3, A9, A12**).

The facilitator also noted whether the participant was able to complete the task steps. Categorizing the step as completed, not completed, or partial if there are some missing parts but the base idea works.

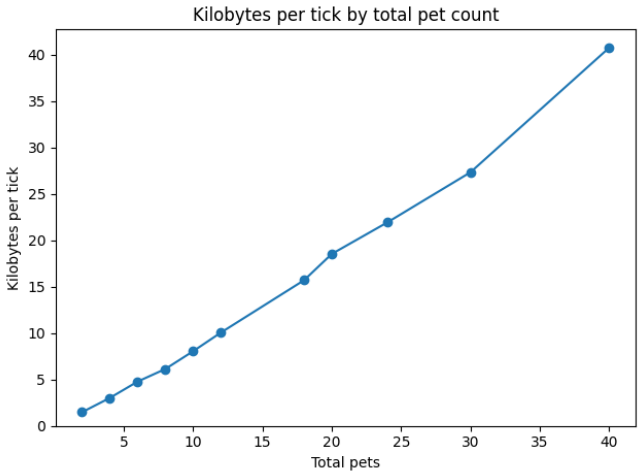


Fig. 4. Network usage scenario 1: number of pets is linearly proportional to the number of pets and does not depend on the number of environments they are distributed across.

V. Results

A. Network usage

The network usage as per **Figure 4** is dependent only on the number of pets and increases linearly. Changing the total number of environments, or number of pets per environment, has no visible effect on the network usage as long as the total number of pets stays the same.

B. Extension user study

The study had a total of 9 participants; the data for 2 participants were excluded for the second part of the task on account of one of them being a pilot study and the second one not having a stable internet connection for task 2. Their insights and suggestions on extension and use case scenarios are still reported in this section. The **Figure 5** shows the results of the Likert scale questionnaire. 2 participants chose the invite task, 2 mating and 3 chose personality task.

Network and federation view is difficult to understand: Participants showed considerable difficulty when working with the API and Federation View, taking more time to think about their approach or stating that they were unsure of how to do that sub-task. Participants who had some web-development experience (3/7) did not find the API or the federation view confusing. This reflected them either assuming they didn't even try to extend the API (2/7) or disagreeing with its extensibility(2/7).

Extensibility Participants found the tasks simple and could place where their code fits into the framework. They generally found the pet, environment, and activity models easy to extend; 5/7 participants agreed that they extended the models easily, and 5/7 participants agreed that they could create new models. Responses concerning the extensibility of pet Systems were more neutral. Generally, the participants found the framework learnable, understandable, usable, and the framework's abstractions useful. **Abstraction**: All participants found the

functions useful; most participants felt that they did not need to know background knowledge to do the tasks (4/7). **Learnability**: Most participants felt that the task got easier as they looked at more of the framework (4/7), and 2/7 felt neutral about it. While 2 participants felt they had to write more code than they initially expected, the other 5 felt neutral about it. **Usability**: Most participants found that the framework didn't take long to navigate(5) and that they would be able to implement the code completely without further help. They also mentioned that they would want to use this in their own projects to make virtual pets. **Understandability**: Most participants felt that the architecture was easy to understand, didn't take a long time to parse, and the flow of code was easy. (5/7)

C. Themes from the interview and tasks

The interview with the developers led to interesting ways in which they want to try to use the framework. The relevant subjective feedback and answers from the task are also included here. The results are presented as per 4 themes that emerged.

1) R1:Increased Simulation Complexity: When presented with the current simple pet and interaction model, the participants wanted there to be more complex models behind the behavior of the pet. The most common suggestion was giving the pets diverse personalities. The participants suggested personality models like having an affinity for certain environments (which they suggested the pets could move around in independently), or other pets based on similarity of their traits. They also suggested an implementation of common personality models like MBTI [30] or traits that can be used on a linear scale such as "Chatty". As part of their extensions, these traits would directly affect their affinity for activities (based on activity tags that they expected to assign to each activity) or other pets.

Another part of the simulation that came up often as an extension was associating events with activities. In this case, the participants wanted activities ending to trigger events, such as a talk or reading activity "increasing knowledge" (assuming a knowledge stat for some category), or an activity like "fight" could result in the death of the pet. Some participants also suggested using the events attached to activities to further the simulation and treating environments as a "prey-predator simulation environment"; they suggest that the participant could see Multi-Agent System like experiments running on the framework.

2) R2:Connection to Existing Social Platforms: Fediflock concept of social pets was particularly attractive to some participants. Some participants suggested that they would like to integrate their pet (or server) with existing social platforms like Letterboxd or Goodreads[5]. They mentioned that being able to let your "movie fan" pet interact with other movie fan pets and exchange information about the movies would let the user get movie suggestions from the user's own social circle. These suggestions came with a discussion about a need for replacement of suggestion algorithms, and wanting a "personalized and controllable but still autonomous and social algorithm". Another similar suggestion was Fediverse integration, allowing the user to post "updates" of interesting things the pet does. In a similar discussion of a user's habit tracking methods, they suggested using Habitica's pets with Fediverse social capabilities might make having the Habitica pets more interesting. Another interesting "plugin" like suggestion was to treat the virtual pet like Clippy [31]. Where the user would ask the virtual pet to help with the environment they are currently working in, most suggestions included use of LLMs for this.

[5]Letterboxd and Goodreads are social platforms for book rating.

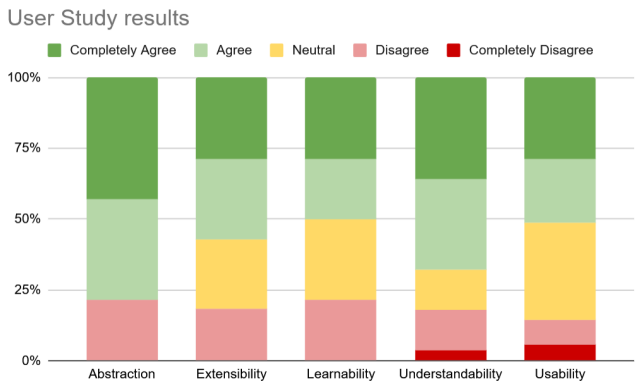


Fig. 5. Results of the Likert scale questionnaire. The diagram shows aggregate results as per categories. Abstraction is most prominently reported positively, while usability showed many neutral answers, possibly due to most participants not using the entire system to comment on it.

3) R3:New Devices and Environments: An interesting direction of suggestion was towards other devices or front ends. Looking at the front-end implementation, some participants suggested making a different front end: a desktop pet or a mobile version. They also mentioned they would want to build a physical version similar to Tamagotchi, or make mobile widgets to see what the pet and the pet's friends are up to on their home screen. Some participants also suggested treating their device as an environment, for pets to be able to interact within the device itself, and other pets being able to "move" to their device's environment.

The participants also mentioned wanting to make offline virtual pets and connect to the internet when available for mobile devices. They then started suggesting ways they would want to synchronize the servers and communicate to the external world when needed, and the internet was available. Another such suggestion was to have situated pet environments as servers; a participant suggested using a location like an office or a cafe and putting their pet in a key-chain that they could put near the entrance and letting all pets in that keychain rack interact with each other (treating the keychain rack as an environment in this case).

VI. Discussion

A. Interpreting the results

The extension user study highlighted that the abstractions do help a developer but also indicate that a non-experienced

developer may have difficulty understanding or extending the API the federation views. This may scare or restrict newer developers from engaging with the framework entirely. Although using a statically typed language like TypeScript did allow some participants to infer what to do and "just follow what the code is already doing". This reliance on code structure in stead of the understanding of the how the API works may result in frustrations when the code doesn't work.

The interviews and observations also suggest that many developers want to implement their own personality model. Result **R1** suggests that including a personality model in the framework itself may hence feel restrictive. While the fact that activities could have events could lead to more interesting interpretations, like "sharing behavior" where a server could "teach" another server some behaviors like "mating", borrowing code from the source server. This would of course come with security issues. Result **R2** points towards a want for knowing what the user's friends are doing without directly interacting with the user, and towards more functional use cases of the pets. Another interpretation of connecting to a social platform could be to treat the platform as a pet environment.

Result **R3** shows that developers would want to engage with the framework outside websites. The physically present yet social virtual pet that occasionally connects to the internet could allow a user to be more "offline" while still being up to date with their friends, similar to how Locket [6] allows users to see their friends on the home screen and updates whenever it can. Another interesting direction with R3 was how the developers re-imagined environments for the pets. Environments in Fediflock are designed to be interpreted open-endedly, so it made sense to see such diverse interpretations.

B. Challenges and reflections on the development process

Developing the framework exposed several challenges that limit the current implementation.

1) Federation: One of the main issues after establishing a base simulation was how I would make the communication decentralized. My first attempts explored using ActivityPub and implementing my framework on top of that, but as mentioned in section II-D, constant communication with ActivityPub is not only expensive, but it is also difficult to implement the protocol from scratch. After a few attempts, I realized I didn't need my framework to work with microblogging or social media platforms, as essentially it wasn't designed for mass communication with other humans (instead, it is supposed to have them with other systems). So it seemed overkill to use ActivityPub, and if I was going to have to implement the whole system anyway, it might be easier to build a simpler federation system myself.

The final implementation uses a simpler federating mechanism based on HTTP API routes and Federation Views. The resulting system is more closely matched to the requirements of the simulation, primarily abstraction and extensibility. This implementation's decoupling of the network communication and the simulation allows for possibility of using popular protocols like activity-pub while keeping the abstractions in the simulation the same.

2) Perfect information in decentralized systems: A decentralized system cannot guarantee that every participant has a perfect view of the complete system. Due to possibly having fragile servers, it would be difficult to have perfect information for all these systems while maintaining a decentralized model. Eventually I settled on a simpler model that waits for one response for each server and just assumes any timeout, failures, or responses taking too long are causes for that communication to be dropped. I chose to have imperfect information in exchange for persistent information.

C. Research Questions

The findings provide preliminary answers to the three research questions:

RQ1: How do we model virtual pets for autonomous behaviour and inter-pet interactions? Fediflock models virtual pets as autonomous and persistent agents situated in environments. A pet's behaviour is divided into modular systems responsible for decision-making, simulation, and activities. The environment provides information about available items and other pets, while activities provide a mechanism for representing pets' actions and interactions with other pets. The developer study suggests that this model is sufficiently understandable, learnable, and usable for developers to use the framework.

RQ2: How do we design decentralized interaction strategies for virtual pet communication? The framework uses HTTP API endpoints together with Federation Views to expose remote pets, environments, and activities. Federation Views represent remote objects through local abstractions, allowing the simulation objects to interact with remote entities without making any direct HTTP calls at each communication point. The communication model is asynchronous and accepts incomplete information and failures as part of the design. The development process showed that using activities as first-class objects gives a singular shared representation of activity even on inconsistent server models. While practical, this method is still limited in its consistency and dependency on good network connections.

RQ3: How do developers want to use and extend a framework for virtual pets and decentralized communication? The developer study indicates that participants were interested in extending the framework in several directions, particularly more complex pet simulation, social platform integrations, and porting to new devices. Most participants were able to understand the general architecture and proposed extensions that made use of the framework's abstractions.

Thus, the results indicate that the framework provides a useful starting point for developers interested in constructing autonomous virtual pets and related multi-agent systems.

[6]Locket is a smartphone widget

VII. Conclusion

The thesis explored how virtual pets can be designed as autonomous and persistent software agents capable of interacting with other pets across decentralized systems. Existing virtual pets commonly provide users with a direct relationship with an individual creature, while interactions between pets are often constrained by the application or platform in which those pets exist. This work investigated an alternative model in which pets can independently exist, discover other pets, and communicate across independently operated servers.

The scope of the framework and the user study highlighted several limitations. The personality model designed during the work was not explored as an implementation or prototype. The communication model needs to incorporate security, a consistency model, robustness measures, and optimize network usage. The current framework also omits saving and loading data when the server is offline. Finally, the developer study provides an initial evaluation of the framework, but a more comprehensive study would require more participants and longer semi-structured interviews with designers.

These limitations provide a clear direction for future work. Some other future directions could include support integration with Fediverse and offline operations. A survey to study the games surrounding current virtual pet/desktop pet culture would also help inform future directions.

Broadly, the work suggests that virtual pets can serve as a useful domain for not only studying autonomous multi-agent systems but also exploring social behaviors. When pets are given the ability to persist, make decisions, inhabit environments, and interact with other autonomous pets, they can easily become means to explore newer methods of interaction and navigating the online social media landscape. The Fediflock framework provides a foundation to let users and developers combine the concepts of virtual pets with decentralized communication.

Perhaps the next step is to show the caged birds that there are other worlds out there.

References

[1] E. Akira, "Neko: History of a Software Pet." [Online]. Available: https://eliotakira.com/neko/

[2] L.-R. BLOCH and DAFNA. LEMISH, "Disposable Love: The Rise and Fall of a Virtual Pet," *New Media & Society*, vol. 1, no. 3, pp. 283–303, Dec. 1999. [Online]. Available: https://doi.org/10.1177/14614449922225591

[3] J. Wrye, "Beyond Pets: Exploring Relational Perspectives of Petness," *Canadian Journal of Sociology*, vol. 34, no. 4, pp. 1033–1064, May 2009. [Online]. Available: https://journals.library.ualberta.ca/cjs/index.php/CJS/article/view/4110

[4] K. Johnsen, S. J. Ahn, J. Moore, S. Brown, T. P. Robertson, A. Marable, and A. Basu, "Mixed Reality Virtual Pets to Reduce Childhood Obesity," *IEEE Transactions on Visualization and Computer Graphics*, vol. 20, no. 4, pp. 523–530, Apr. 2014. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/6777460

[5] J. Hellings, P. Leek, and B. Bredeweg, "StudyGotchi: Tamagotchi-Like Game-Mechanics to Motivate Students During a Programming Course," in *Transforming Learning with Meaningful Technologies*, M. Scheffel, J. Broisin, V. Pammer-Schindler, A. Ioannou, and J. Schneider, Eds. Cham: Springer International Publishing, 2019, pp. 622–625.

[6] "Pokémon Sleep," The Pokémon Company, Oct. 2026. [Online]. Available: https://www.pokemonsleep.net/en

[7] "Pets," *Habitica Wiki*. [Online]. Available: https://habitica.fandom.com/wiki/Pets

[8] T. M. Graham and T. D. Glover, "On the Fence: Dog Parks in the (Un)Leashing of Community and Social Capital," *Leisure Sciences*, vol. 36, no. 3, pp. 217–234, May 2014. [Online]. Available: https://doi.org/10.1080/01490400.2014.888020

[9] D. D. Dumitrica, "An exploration of cheating in a virtual gaming world," *Journal of Gaming and Virtual Worlds*, vol. 3, no. 1, pp. 21–36, Mar. 2011. [Online]. Available: https://www.scopus.com/pages/publications/85055461005

[10] M. Charity, M. Cook, and N. Vas, "New Frontiers in AI for Game Design (Dagstuhl Seminar 25292)," *Dagstuhl Reports*, vol. 15, no. 7, 2026. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/DagRep.15.7.124

[11] O. Mival, S. Cringean, and D. Benyon, "Personification technologies: Developing artificial companions for older people," Jan. 2004.

[12] M. Dodd, A. Fowler, and D. Lottridge, "Purr-ogrammed love: A narrative review of virtual pets," *Entertainment Computing*, vol. 54, p. 100958, Jun. 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1875952125000382

[13] A. Rogge, "Defining, Designing and Distinguishing Artificial Companions: A Systematic Literature Review," *International Journal of Social Robotics*, vol. 15, no. 9, pp. 1557–1579, Oct. 2023. [Online]. Available: https://doi.org/10.1007/s12369-023-01031-y

[14] D. Bylieva, N. Almazova, V. Lobatyuk, and A. Rubtsova, "Virtual Pet: Trends of Development," in *Digital Science 2019*, T. Antipova and Á. Rocha, Eds. Cham: Springer International Publishing, 2020, pp. 545–554.

[15] S. M. Grimes and L. R. Shade, "Neopian economics of play: Children's cyberpets and online communities as immersive advertising in NeoPets.com," *International Journal of Media & Cultural Politics*, vol. 1, no. 2, pp. 181–198, Sep. 2005. [Online]. Available: https://intellectdiscover.com/content/journals/10.1386/macp.1.2.181/1

[16] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, Jun. 1995. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0269888900008122/type/journal_article

[17] A. Dorri, S. S. Kanhere, and R. Jurdak, "Multi-Agent Systems: A Survey," *IEEE Access*, vol. 6, pp. 28 573–28 593, 2018. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8352646/references

[18] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley Publishing, Jun. 2009.

[19] T. Finin, R. Fritzson, D. McKay, and R. McEntire, "KQML as an agent communication language," in *Proceedings of the Third International Conference on Information and Knowledge Management*, ser. CIKM '94. New York, NY, USA: Association for Computing Machinery, Nov. 1994, pp. 456–463. [Online]. Available: https://dl.acm.org/doi/10.1145/191246.191322

[20] "A2A Protocol." [Online]. Available: https://a2a-protocol.org/latest/

[21] A. Datta, S. Buchegger, L.-H. Vu, T. Strufe, and K. Rzadca, "Decentralized Online Social Networks," in *Handbook of Social Network Technologies and Applications*, B. Furht, Ed. New York, NY: Springer US, 2010, pp. 349–378. [Online]. Available: https://doi.org/10.1007/978-1-4419-7142-5_17

[22] L. La Cava, S. Greco, and A. Tagarelli, "Understanding the growth of the Fediverse through the lens of Mastodon," *Applied Network Science*, vol. 6, no. 1, p. 64, Sep. 2021. [Online]. Available: https://doi.org/10.1007/s41109-021-00392-5

[23] "ActivityPub." [Online]. Available: https://www.w3.org/TR/activitypub/

[24] "AT Protocol." [Online]. Available: https://atproto.com

[25] M. F, "Matrix: An open network for secure, decentralised communication." [Online]. Available: https://matrix.org/

[26] J. A. P. Martins, P. A. L. Rego, J. A. F. de Macêdo, F. A. Silva, and V. Lagrota, "Matrix protocol: A comprehensive systematic mapping study," *Journal of Cloud Computing*, vol. 15, no. 1, p. 20, Feb. 2026. [Online]. Available: https://doi.org/10.1186/s13677-025-00829-7

[27] A. Corsaro, L. Cominardi, O. Hecart, G. Baldoni, J. E. P. Avital, J. Loudet, C. Guimares, M. Ilyin, and D. Bannov, "Zenoh: Unifying Communication, Storage and Computation from the Cloud to the Microcontroller," in *2023 26th Euromicro Conference on Digital System Design (DSD)*, Sep. 2023, pp. 422–428. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10456820

[28] T. Tullis and B. Albert, "Chapter 6 - Self-Reported Metrics," in *Measuring the User Experience (Second Edition)*, ser. Interactive

Technologies. Boston: Morgan Kaufmann, Jan. 2013, pp. 121–161. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780124157811000066

[29] M. Piccioni, C. A. Furia, and B. Meyer, "An Empirical Study of API Usability," in *2013 ACM / IEEE International Symposium on Empirical Software Engineering and Measurement*. Baltimore, Maryland: IEEE, Oct. 2013, pp. 5–14. [Online]. Available: http://ieeexplore.ieee.org/document/6681333/

[30] K. C. Briggs and I. B. Myers, *The myers-briggs type indicator: Form G*. Consulting Psychologists Press, 1977.

[31] Microsoft, "Microsoft Clippy," Aug. 2001. [Online]. Available: https://web.archive.org/web/20010811130237/http://www.microsoft.com/office/clippy/

APPENDIX A
USER STUDY : TASKS

A. Invite

The pets want to move to the modern world and communicate 'online'. Can you give them the ability to have contacts on their phone that they can use to invite friends over?

1) Add to the API to have an invite endpoint for pets, this endpoint will receive the invite request from another pet. If the pet accepts then the pet should move to the environment it has been invited to.
2) Update the pet model to store contacts of pets it has done activities with.
3) Make a function that can be called when the pet has low happiness, this function sends an invite to a pet from its contacts that is not in the current environment.
4) Similarly there should be a function to call for the pet inside the pet model to check the invite request and accept/reject that can then return the answer to the endpoint

B. Mating

The pets want to be able to reproduce and be parents to new pets. Can you create a mating center environment for two pets to be able to reproduce?

1) Create a mating center environment, that has an item with the activity 'mate', that can be done with 2 pets only.
2) You can have any criteria for the pet to accept or reject that activity, it could be based on stats, relationship. Provide functions for these.
3) If accepted, two pets do the activity. Once the activity ends, there should be a third pet created in the environment. This new pet can belong to either of the parent's servers.
4) Add some way to track the parent-child relationships in the pet model. And update your psedo-code above to update this

C. Personality

The pets feel ready to develop complex brains and have personality. Can you give the pets a simple personality system that decides what activities they accept or reject.

1) Update the current pet model and systems to have some sort of personality (check ideas below).
2) Update the Pet System to use that personality to decide if the pet will accept or reject an activity.
3) Create an API point and corresponding functions to access this personality and activity history.

Personality Ideas

- Preference of activity based on stats changes from doing the activity
- Preference of activity based on activity or items tags

APPENDIX B
USER STUDY: POST TASKS QUESTIONNAIRE

On a scale of 1 to 5, where 1 means completely disagree and 5 means completely agree, rate statements in sections A and B in accordance with the task you just did.

A. Task questions

1) I found the tasks simple.
2) Once I solved the first part of the task, I found the other part easier.
3) The amount of code required to solve the task matched my expectations.
4) I felt the need to know related background knowledge to accomplish the task
5) The functions used in the code were useful during my task.
6) The code architecture was simple to understand.
7) It was easy to follow the flow of code between functions
8) I had to look through a lot of files to find pieces of code.
9) I had to look for a long time to find the piece of code I wanted to find.
10) I would be able to implement an extension fully without further help.
11) I could easily place what part of my pseudo code should go where in the system
12) Given time, I would want to use this framework to make a virtual pet server of my own.

B. Code Architecture questions

1) I found using the API was simple
2) I found it easy to extend the API
3) The models were easy to extend
4) It was easy to make a new model.
5) The pet systems were easy to understand.
6) The pet systems were easy to extend.
7) It was easy to make a new pet system.